

A Netlink message is a buffer that holds both the Netlink header and the Netlink payload. The buffer is passed to the Netlink core through *iovec* structure. The structure (*include/uapi/linux/uio.h*) definition is as follows:

```
struct iovec{
void __user *iov_base;
__kernel_size_t iov_len;
};
```

In the above code, *iov_base* holds the base address of the Netlink message buffer, and *iov_len* holds the length of the Netlink message buffer, which is the size of the Netlink header and payload.

Socket messages are sent through the *sendmsg* API, which requires the *msghdr* structure as a parameter. The following fields of *struct msghdr* are useful:

```
struct msghdr
{
void *msg_name;
int msg_namelen;
struct iovec *msg_iov;
__kernel_size_t msg_iovlen;
//other fields not discussed
};
```

In the above code...

- *msg_name* is the base address of the *struct sockaddr_nl* variable, which holds information about the destination address.
- *msg_namelen* is the length of the structure, which is pointed by the *msg_name* field.
- *msg_iov* is the address of the *iovec* structure which holds the netlink message buffer.
- *msg_iovlen* is the length of the netlink message buffer.

Process-to-process unicast communication

Unicast sender example: The following header file needs to be included in an application:

```
#include <sys/socket.h>
#include <linux/netlink.h>
```

First, the application has to create a Netlink socket, which is done through the socket API as follows:

```
int fd = socket(PF_NETLINK, SOCK_RAW, NETLINK_GENERIC);
```

After creating the Netlink socket, the application has to bind the socket with the unique address as follows:

```
struct sockaddr_nl src_addr;
```

```
//AF_NETLINK socket protocol
src_addr.nl_family = AF_NETLINK;
```

```
//application unique id
src_addr.nl_pid = 1;
```

```
//specify not a multicast communication
src_addr.nl_groups = 0;
```

```
//attach socket to unique id or address
bind(fd, (struct sockaddr *)&src_addr, sizeof(src_addr));
```

After binding the socket with the unique address, the application has to define the destination address, message header, message payload, *iovec* structure and send the message using the *sendmsg* API as follows:

```
//total netlink message length
#define NLINK_MSG_LEN 1024
```

```
struct sockaddr_nl dest_addr;

dest_addr.nl_family = AF_NETLINK;
```

```
//destination process id
dest_addr.nl_pid = 2;
```

```
dest_addr.nl_groups = 0;
```

```
//allocate buffer for netlink message which
//is message header + message payload
struct nlmsghdr *nlh =(struct nlmsghdr *) malloc(NLMSG_
SPACE(NLINK_MSG_LEN));
```

Here, *NLMSG_SPACE* is the macro that gives an aligned length for the Netlink message.

```
//netlink message length
nlh->nlmsg_len = NLMSG_SPACE(NLINK_MSG_LEN);
```

```
//src application unique id
nlh->nlmsg_pid = 1;
```

```
nlh->nlmsg_flags = 0;
```

```
//copy the payload to be sent
strcpy(NLMSG_DATA(nlh), "Hello Process");
```

Here, the *NLMSG_DATA* macro is used to access the address of the payload.

```
//fill the iovec structure
struct iovec iov;
```